



المدرسة الوطنية للعلوم التطبيقية - بني ملال
+213 34 80 00 00 | +213 34 80 00 00 - 07 00 00 00
Ecole Nationale des Sciences Appliquées - Béni Mellal

Université Sultan Moulay Slimane
École Nationale des Sciences Appliquées
Béni Mellal

Mémoire de Projet - Module Apprentissage Par Projet

Filière :
Intelligence Artificielle et Cybersécurité

Sous le thème :

**Conception et réalisation d'un chatbot
intelligent pour l'assistance universitaire à
l'ENSA Béni Mellal**

Réalisé par :

**ALLADO KOSSI RICHARD
ADJEVI KODJO MARIUS-BEN**

Encadré par :

Prof. ELBAGHAZAOUI (ENSA Béni Mellal)

Soutenu le 22 Mai 2026 devant le jury composé de :

Membre	Établissement	Qualité
Pr. TOUIL	ENSA Béni Mellal	Rapporteur
Pr. ESSWIDI	ENSA Béni Mellal	Rapporteur
Prof. ELBAGHAZAOUI	ENSA Béni Mellal	Encadrant

Année universitaire : 2025–2026

Avant-propos

Le présent rapport a été réalisé dans le cadre du Projet de Fin d'Année de la filière IACS – Intelligence Artificielle et Cybersécurité – à l'École Nationale des Sciences Appliquées de Béni Mellal. Ce travail académique porte sur la conception et la réalisation d'un chatbot intelligent destiné à accompagner les étudiants dans l'accès aux informations universitaires.

Le projet a été mené par ALLADO KOSSI RICHARD et ADJEVI KODJO MARIUS-BEN, sous l'encadrement de Prof. ELBAGHAZAOU. Il mobilise des notions de développement web, d'intelligence artificielle, de recherche sémantique, de cybersécurité applicative, de gestion de bases de données, de tests et de déploiement.

Résumé

Ce rapport de Projet de Fin d'Année présente la conception et la réalisation de *Chatbot USMS*, un assistant universitaire intelligent destiné aux étudiants de l'ENSA Béni Mellal. Le projet facilite l'accès aux informations académiques courantes : questions fréquentes, emplois du temps, filières, procédures et informations administratives.

La solution repose sur une architecture web moderne composée d'un frontend Next.js, d'une API FastAPI, d'une base PostgreSQL, d'un moteur vectoriel Qdrant, d'un cache Redis et d'un modèle de langage accessible via Groq Cloud. Le système combine une approche RAG pour exploiter les documents FAQ, un routage métier pour traiter les emplois du temps à partir de données structurées, un système de détection des émotions pour adapter le ton des réponses, et des mécanismes de sécurité applicative inspirés des principes du RGPD, à compléter en production marocaine par les exigences de la loi 09-08 et de la CNDP.

Mots clés : chatbot, RAG, intelligence artificielle, cybersécurité, FastAPI, Next.js, PostgreSQL, Qdrant, Groq, ENSA Béni Mellal.

Résumé en arabe

ملال. يبني التطبيقية للعلوم الوطنية المدرسة بطلبة خاص ذكي جامعي مساعد وإنجاز تصميم التقرير هذا يقدم معلومات الحصص، جداول المتداولة، الأسئلة مثل الجامعية المعلومات إلى الولوج تسهيل إلى المشروع يهدف الإدارية. والخدمات الشعب

بيانات وقاعدة، FastAPI باستعمال خلفية وواجهة، Next.js بتقنية أمامية واجهة على النظام يعتمد يدمج كما. Groq Cloud عبر كبير لغوي ونموذج، Qdrant متجهي بحث ومحرك، PostgreSQL وتأمين المدخلات من التحقق الصلاحيات، مراقبة الجلسات، حماية خلال من السيبراني الأمن جوانب المشروع الشخصية. المعطيات حماية مبادئ مراعاة مع الإدارة عمليات

Abstract

This report presents the design and implementation of *Chatbot USMS*, an intelligent academic assistant for ENSA Beni Mellal students. The application simplifies access to university information such as frequently asked questions, timetables, academic tracks and administrative guidance.

The solution is based on a Next.js frontend, a FastAPI backend, a PostgreSQL database, Qdrant vector search, Redis caching and a large language model served through Groq Cloud. It combines Retrieval-Augmented Generation with business rules dedicated to timetable questions, an emotion detection module to adapt response tone, and application security mechanisms inspired by GDPR principles and to be completed, for deployment in Morocco, by Law 09-08 and CNDP requirements for authentication, access control and administrative operations.

Remerciements

Nous tenons tout d’abord à exprimer notre profonde gratitude à Monsieur le Directeur de l’École Nationale des Sciences Appliquées de Béni Mellal, Pr. Belaid BOUIKHALENE, ainsi qu’à Monsieur le Directeur adjoint, Pr. Soufiane BELHOUIDEG, pour les moyens pédagogiques et l’environnement de formation mis à la disposition des étudiants.

Nous adressons également nos sincères remerciements à notre coordinateur de filière, Professeur Mohamed GOUSKIR, pour son accompagnement, son engagement et les efforts fournis afin d’assurer le bon déroulement de la formation au sein de la filière IACS – Intelligence Artificielle et Cybersécurité.

Nous remercions tout particulièrement notre encadrant pédagogique, Prof. ELBAGHA-ZAOUI, pour son suivi, ses orientations, ses remarques constructives et sa disponibilité tout au long de la réalisation de ce Projet de Fin d’Année.

Nos remerciements vont aussi à l’ensemble du corps professoral et administratif de l’ENSA Béni Mellal pour la qualité de l’encadrement et des enseignements reçus durant notre parcours.

Enfin, nous remercions les membres du jury pour l’attention portée à notre travail, ainsi que toutes les personnes ayant contribué, directement ou indirectement, à l’aboutissement de ce projet.

Table des matières

Avant-propos	I
Résumé	II
Résumé en arabe	III
Abstract	IV
Remerciements	V
Sommaire	VI
Liste des figures	VII
Liste des tableaux	VIII
Liste des abréviations	IX
Introduction générale	1
1 Contexte général et cahier des charges	2
1.1 Introduction	2
1.2 Présentation du contexte	2
1.3 Problématique	2
1.4 Solution proposée	2
1.5 Objectifs	3
1.6 Exigences non fonctionnelles	3
1.7 Périmètre fonctionnel	4
1.8 Conclusion	4
2 Analyse et conception de la solution	5
2.1 Introduction	5
2.2 Architecture générale	5
2.3 Acteurs	6
2.4 Modèle de données	7
2.5 Traitement d'une question	8
2.6 Choix technologiques	8
2.7 Conclusion	10

3	Réalisation et implémentation	11
3.1	Introduction	11
3.2	Backend FastAPI	11
3.2.1	Organisation et initialisation	11
3.2.2	Pile de middleware	11
3.2.3	Sondes de disponibilité	12
3.3	Moteur RAG	12
3.3.1	Vue d'ensemble	12
3.3.2	Sources documentaires	12
3.3.3	Pipeline de récupération	12
3.3.4	Évaluation de la confiance	13
3.3.5	Gestion des documents FAQ	13
3.4	Intégration LLM – Groq Cloud	13
3.4.1	Usages du modèle	13
3.4.2	Masquage des données personnelles	14
3.4.3	Protection contre l'injection de prompt	14
3.5	Routage et détection des émotions	14
3.5.1	Routage des questions	14
3.5.2	Détection des émotions	14
3.6	Stratégie de cache à deux couches	15
3.7	Gestion des emplois du temps	15
3.8	Frontend Next.js	17
3.8.1	Architecture frontend	17
3.8.2	Streaming et interface de chat	17
3.8.3	Internationalisation	18
3.9	Axe cybersécurité	20
3.9.1	Objectifs de sécurité	20
3.9.2	Authentification et gestion des sessions	20
3.9.3	Protection contre le force brute	21
3.9.4	Politique de mots de passe	21
3.9.5	Contrôle d'accès basé sur les rôles (RBAC)	21
3.9.6	Protection des données	22
3.9.7	Limites et durcissement recommandé	22
3.10	Administration	23
3.11	Migration PostgreSQL	26
3.12	Outillage	26
3.13	Conclusion	27
4	Tests, déploiement et évaluation	28

4.1	Introduction	28
4.2	Tests automatisés	28
4.3	Pipeline d'intégration continue	29
4.4	Scénarios de validation fonctionnelle	29
4.5	Déploiement local	29
4.6	Déploiement Docker	29
4.7	Évaluation	31
4.8	Conclusion	32
	Conclusion générale	33
	Bibliographie	34
	Annexes	35
.1	Arborescence simplifiée du projet	35
.2	Livrables du projet	35
.3	Résumé des endpoints API principaux	36

Table des figures

2.1	Architecture globale de la solution	5
2.2	Cycle de traitement d'une question	8
3.1	Chaîne de traitement RAG	12
3.2	Affichage structuré d'un emploi du temps dans le chatbot	16
3.3	Page de connexion sécurisée de l'application	17
3.4	Interface principale du chatbot côté étudiant	18
3.5	Exemple de réponse issue de la base de connaissances FAQ	18
3.6	Page profil étudiant	19
3.7	Module de planification des révisions	19
3.8	Tableau de bord administrateur	23
3.9	Aperçu d'un document FAQ avant action administrateur	24
3.10	Gestion de la base FAQ dans l'espace administrateur	25
3.11	Gestion des emplois du temps publiés	25
3.12	Gestion des utilisateurs et des demandes de filière	26
4.1	Documentation Swagger de l'API FastAPI	31

Liste des tableaux

1.1	Exigences non fonctionnelles retenues	3
1.2	Périmètre fonctionnel du projet	4
2.1	Services backend et leurs responsabilités	6
2.2	Récapitulatif des responsabilités par acteur	7
2.3	Entités principales de la base de données	7
2.4	Récapitulatif des technologies par couche	9
2.5	Détails IA et recherche sémantique réellement implémentés	10
3.1	En-têtes de sécurité HTTP appliqués à chaque réponse	11
3.2	Niveaux de confiance RAG	13
3.3	États émotionnels détectés et adaptation du ton	15
3.4	Conditions de contournement du cache	15
3.5	Séparation entre LLM seul, RAG et données structurées	16
3.6	Objectifs de sécurité du projet	20
3.7	Paramètres de configuration des jetons JWT	21
3.8	Menaces principales et mesures appliquées	22
3.9	Fonctions disponibles dans le tableau de bord administrateur	24
3.10	Scripts utilitaires du projet	27
4.1	Modules de tests et couverture	28
4.2	Variables d'environnement requises pour le déploiement	30
4.3	Bilan d'évaluation par objectif	31
4	Principaux endpoints de l'API REST	36

Liste des abréviations

Abréviation	Signification
API	Application Programming Interface
CI/CD	Continuous Integration / Continuous Deployment
CORS	Cross-Origin Resource Sharing
CSRF	Cross-Site Request Forgery
EDT	Emploi du temps
ENSA BM	École Nationale des Sciences Appliquées de Béni Mellal
FAQ	Foire aux questions
GDPR/RGPD	General Data Protection Regulation / Règlement Général sur la Protection des Données
HMAC	Hash-based Message Authentication Code
HSTS	HTTP Strict Transport Security
IACS	Intelligence Artificielle et Cybersécurité
IDOR	Insecure Direct Object Reference
JWT	JSON Web Token
LLM	Large Language Model
LOC	Lines of Code
PFA	Projet de Fin d'Année
PII	Personally Identifiable Information
RAG	Retrieval-Augmented Generation
RBAC	Role-Based Access Control
RTL	Right-to-Left
SSE	Server-Sent Events
TTL	Time To Live
USMS	Université Sultan Moulay Slimane

Introduction générale

La transformation digitale des établissements universitaires vise à rendre les services académiques plus accessibles, plus rapides et plus cohérents. Dans ce contexte, les assistants conversationnels représentent une solution adaptée pour répondre aux questions répétitives des étudiants et centraliser l'accès à l'information.

À l'ENSA Béni Mellal, les étudiants ont régulièrement besoin d'informations sur les filières, les procédures, les contacts ou les emplois du temps. Ces informations peuvent être dispersées dans différents supports, ce qui rend leur consultation lente. Le projet *Chatbot USMS* répond à cette problématique par une application web conversationnelle capable de comprendre une demande en langage naturel, d'identifier la bonne source d'information et de présenter une réponse claire, dans la langue de l'étudiant.

Ce rapport expose le contexte du projet, l'analyse du besoin, la conception technique, la réalisation, les tests, le déploiement et les perspectives d'amélioration. Il est structuré en quatre chapitres : le cahier des charges, la conception de la solution, l'implémentation, puis la validation et le déploiement.

Chapitre I

Contexte général et cahier des charges

I. Introduction

Ce chapitre présente le cadre général du projet *Chatbot USMS*, la problématique traitée, les objectifs retenus et le périmètre fonctionnel de la solution. Il constitue le point de départ de l'analyse en définissant précisément le besoin auquel le projet répond, les acteurs concernés et les contraintes fonctionnelles et non fonctionnelles qui ont guidé la conception. La lecture de ce chapitre permet de comprendre pourquoi chaque choix technique décrit dans les chapitres suivants a été effectué.

II. Présentation du contexte

L'École Nationale des Sciences Appliquées de Béni Mellal, relevant de l'Université Sultan Moulay Slimane, forme des ingénieurs dans plusieurs spécialités, dont la filière IACS : Intelligence Artificielle et Cybersécurité. Le projet s'inscrit dans une logique de digitalisation des services étudiants, de centralisation des informations utiles et d'application concrète des compétences en IA et en sécurité des systèmes web. Il constitue également une démonstration pratique des technologies modernes de traitement du langage naturel appliquées à un contexte institutionnel réel.

III. Problématique

Les informations universitaires sont disponibles mais souvent réparties entre plusieurs documents ou interfaces. Les difficultés principales sont la répétition des mêmes questions auprès des services administratifs, la recherche manuelle dans des fichiers FAQ, la consultation des emplois du temps par filière et semestre, et la mise à jour régulière du contenu. Cette dispersion entraîne une perte de temps pour les étudiants et une charge répétitive pour le personnel.

IV. Solution proposée

La solution proposée consiste à centraliser l'accès aux informations académiques dans une application web unique. L'étudiant interagit avec un chatbot capable de répondre en langage naturel dans trois langues (français, anglais, arabe), tandis que l'administrateur dispose d'un espace de pilotage pour contrôler les contenus, visualiser les fichiers et maintenir les données de référence. Le projet associe assistance étudiante, gestion documentaire, administration, intelligence artificielle, sécurité applicative, protection des données et traçabilité.

V. Objectifs

L'objectif principal est de réaliser un chatbot universitaire intelligent. Les objectifs spécifiques sont :

- offrir une interface de discussion simple et responsive, accessible en français, en anglais et en arabe ;
- répondre aux questions fréquentes à partir de documents institutionnels via une approche RAG ;
- gérer les questions liées aux emplois du temps par filière et semestre à partir de données structurées ;
- détecter l'état émotionnel de l'étudiant pour adapter le ton des réponses ;
- permettre l'inscription et la connexion sécurisées avec restriction aux adresses institutionnelles ;
- appliquer des mesures de cybersécurité et de protection des données adaptées à une application web universitaire ;
- conserver l'historique des conversations et permettre le feedback sur les réponses ;
- fournir un tableau de bord administrateur pour gérer les FAQ, les utilisateurs, les statistiques et les emplois du temps.

VI. Exigences non fonctionnelles

TABLE 1.1 – Exigences non fonctionnelles retenues

Exigence	Application dans le projet
Sécurité	Sessions par cookies HttpOnly, jetons JWT à courte durée de vie, protection CSRF, contrôle des rôles RBAC, validation des entrées, limitation de débit et séparation des privilèges.
Protection des données	Masquage des données personnelles avant envoi au LLM, droit à l'effacement, droit à la portabilité, journalisation des actions et consentement explicite à l'inscription.
Fiabilité	Données persistées dans PostgreSQL, migrations Alembic versionnées et tests automatisés sur les routes critiques.
Maintenabilité	Séparation frontend/backend, services spécialisés, typage TypeScript strict et scripts de démarrage automatisés.
Performance	Streaming SSE des réponses, cache Redis à deux couches et recherche vectorielle Qdrant pour réduire les traitements répétitifs.

VII. Périmètre fonctionnel

TABLE 1.2 – Périmètre fonctionnel du projet

Module	Description
Authentification et sécurité	Inscription avec consentement et confidentialité, connexion, session JWT, cookies sécurisés, protection CSRF, verrouillage de compte après 5 tentatives échouées, et rôles étudiant/admin.
Chatbot	Questions en langage naturel, streaming SSE, historique de conversation, feedback, détection des émotions et adaptation du ton de réponse.
FAQ intelligente	Recherche vectorielle sémantique dans les documents FAQ et génération contextualisée avec attribution de source.
Emplois du temps	Publication par semestre et filière, consultation structurée et interrogation en langage naturel depuis le chatbot.
Administration	Statistiques, gestion des utilisateurs, FAQ (import, aperçu, suppression, reconstruction d'index), cache, emplois du temps publiés, demandes de changement de filière et questions à améliorer.
Profil étudiant	Langue préférée, filière, activité, export des données, suppression de compte et planning de révision personnel.

VIII. Conclusion

Le besoin identifié dépasse une simple consultation de documents. La solution doit combiner données structurées, documents institutionnels, dialogue naturel multilingue et mesures de sécurité robustes. L'ensemble de ces exigences – fonctionnelles, non fonctionnelles et de sécurité – constitue le socle sur lequel repose la conception du système. Le chapitre suivant présente les choix d'architecture, le modèle de données et les technologies retenues pour répondre à ce cahier des charges.

Chapitre II

Analyse et conception de la solution

I. Introduction

La conception repose sur une séparation claire entre l'interface utilisateur, l'API métier, la persistance, le moteur vectoriel et le service d'intelligence artificielle. Ce chapitre décrit l'architecture globale retenue, les acteurs du système, le modèle de données et les choix technologiques justifiés. Chaque décision de conception vise à garantir la modularité, la maintenabilité et la sécurité de la solution tout en restant réalisable dans le cadre d'un projet académique. Les diagrammes et tableaux présentés ici servent de référence pour comprendre l'implémentation décrite au chapitre suivant.

II. Architecture générale

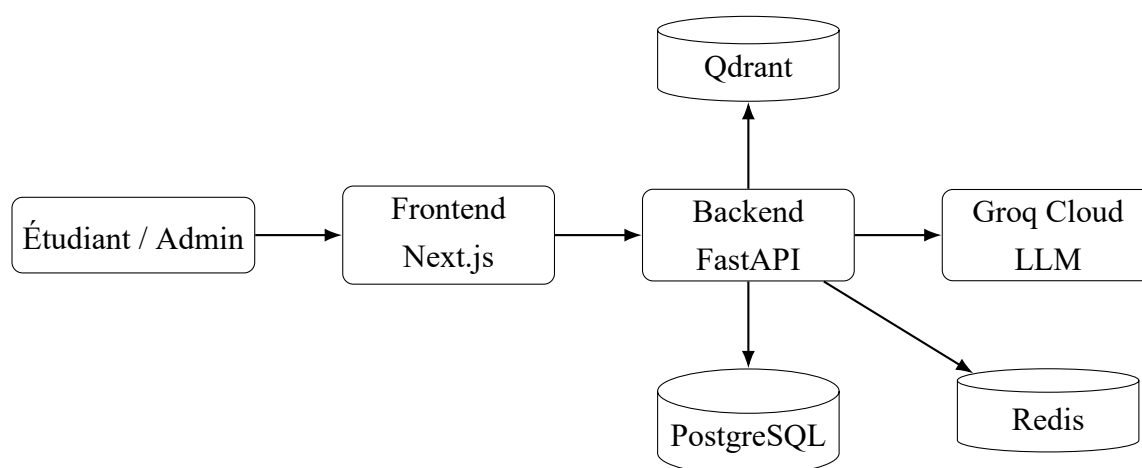


FIGURE 2.1 – Architecture globale de la solution

Le backend FastAPI centralise toute la logique métier et est organisé en services spécialisés. La séparation est stricte : le frontend ne contient aucun secret et ne manipule pas directement les données persistées. Toutes les opérations sensibles transitent par l'API, qui applique les contrôles d'accès avant d'agir.

Le projet, en version documentaire 2.0.0, supporte trois langues (français, anglais et arabe) et repose sur PostgreSQL 15 avec deux collections vectorielles Qdrant : `faq_campus` pour les documents indexés et `semantic_cache` pour les réponses mises en cache. Il compte 9 tables en base de données, plus de 30 endpoints API et plus de 6 000 lignes de code réparties entre le backend Python (~3 900 lignes) et le frontend TypeScript (~2 100 lignes). La suite de tests couvre 127 scénarios automatisés. La durée maximale d'une requête chat est fixée à 90 secondes et la politique de rétention des données par défaut est de 180 jours avant nettoyage.

manuel.

TABLE 2.1 – Services backend et leurs responsabilités

Service	Responsabilité
auth_service	Création et validation des jetons JWT, dépendances FastAPI pour la sécurisation des routes.
db_service	Toutes les opérations CRUD sur PostgreSQL via SQLAlchemy.
router_service	Routage des questions vers le chemin EDT ou FAQ selon une analyse heuristique et résolution floue des codes de filière.
chatbot_service	Détection des émotions, reformulation des données d’emploi du temps, adaptation du ton.
rag_service	Recherche vectorielle Qdrant, embeddings FastEmbed, prétraitement multilingue des requêtes et cache sémantique.
groq_service	Appels LLM, masquage des données personnelles, protection contre l’injection de prompt, génération des titres de conversation.
cache_service	Lecture/écriture Redis, invalidation et cache sémantique à deux couches.
env_service	Validation et chargement des variables d’environnement au démarrage.
logging_service	Configuration Loguru, journalisation structurée vers fichier et console.
limiter_service	Configuration SlowAPI pour la limitation de débit des routes sensibles.

III. Acteurs

Le système distingue deux acteurs. L’étudiant crée un compte institutionnel, choisit sa filière, discute avec le chatbot, consulte et supprime son historique, donne son avis sur les réponses, exporte ses données personnelles et gère son profil. L’administrateur supervise la plateforme : il gère les utilisateurs, importe et supprime les fichiers FAQ, reconstruit l’index vectoriel, vide le cache Redis, publie les emplois du temps par filière et semestre, traite les demandes de changement de filière et consulte les statistiques d’utilisation.

TABLE 2.2 – Récapitulatif des responsabilités par acteur

Acteur	Actions principales
Étudiant	S’inscrire avec consentement et confidentialité, se connecter, poser des questions, consulter ses conversations, réagir aux réponses, demander un changement de filière, gérer sa langue préférée, exporter ou supprimer ses données.
Administrateur	Consulter les statistiques d’usage, gérer les comptes, traiter les demandes étudiantes, importer ou supprimer les FAQ, reconstruire la base vectorielle, publier les emplois du temps, nettoyer les données anciennes et vider le cache.

IV. Modèle de données

Le backend utilise SQLAlchemy avec PostgreSQL. Les migrations de schéma sont gérées par Alembic pour garantir la traçabilité des évolutions. Les principales entités sont présentées ci-dessous.

TABLE 2.3 – Entités principales de la base de données

Entité	Rôle et colonnes clés
users	Compte utilisateur : email (unique), mot de passe haché bcrypt, rôle, filière, langue préférée, compteur de tentatives de connexion échouées et expiration du verrouillage.
conversations	Regroupement des messages d’un utilisateur : titre généré par IA, horodatage.
messages	Tour de parole Q/R : rôle, contenu, fichier source, feedback (-1/0/1), métadonnées JSON (confiance RAG, emploi du temps).
audit_logs	Traçabilité RGPD : action, identifiant utilisateur, horodatage, détails.
timetables	En-tête d’emploi du temps : filière, année universitaire, semestre, statut actif.
timetable_slots	Créneau horaire : jour (0=lundi), heure début/fin, matière, professeur, salle, type (Cours/TD/TP).
qa_review	File d’attente des questions à améliorer détectées par feedback négatif.
class_change_requests	Demande de changement de filière : filière actuelle, filière souhaitée, motif, statut, note de décision.
revision_plans	Plannings de révision personnalisés sauvegardés par l’étudiant.

Le schéma applique des suppressions en cascade : la suppression d’un utilisateur entraîne la suppression de toutes ses conversations, messages, journaux d’audit et demandes associées, conformément au droit à l’effacement du RGPD. La compatibilité de hachage est assurée sur trois formats – bcrypt moderne, bcrypt hérité et SHA-256 – avec migration automatique vers bcrypt lors de la prochaine connexion réussie.

V. Traitement d’une question

Le traitement d’un message étudiant suit un enchaînement précis, représenté ci-dessous.

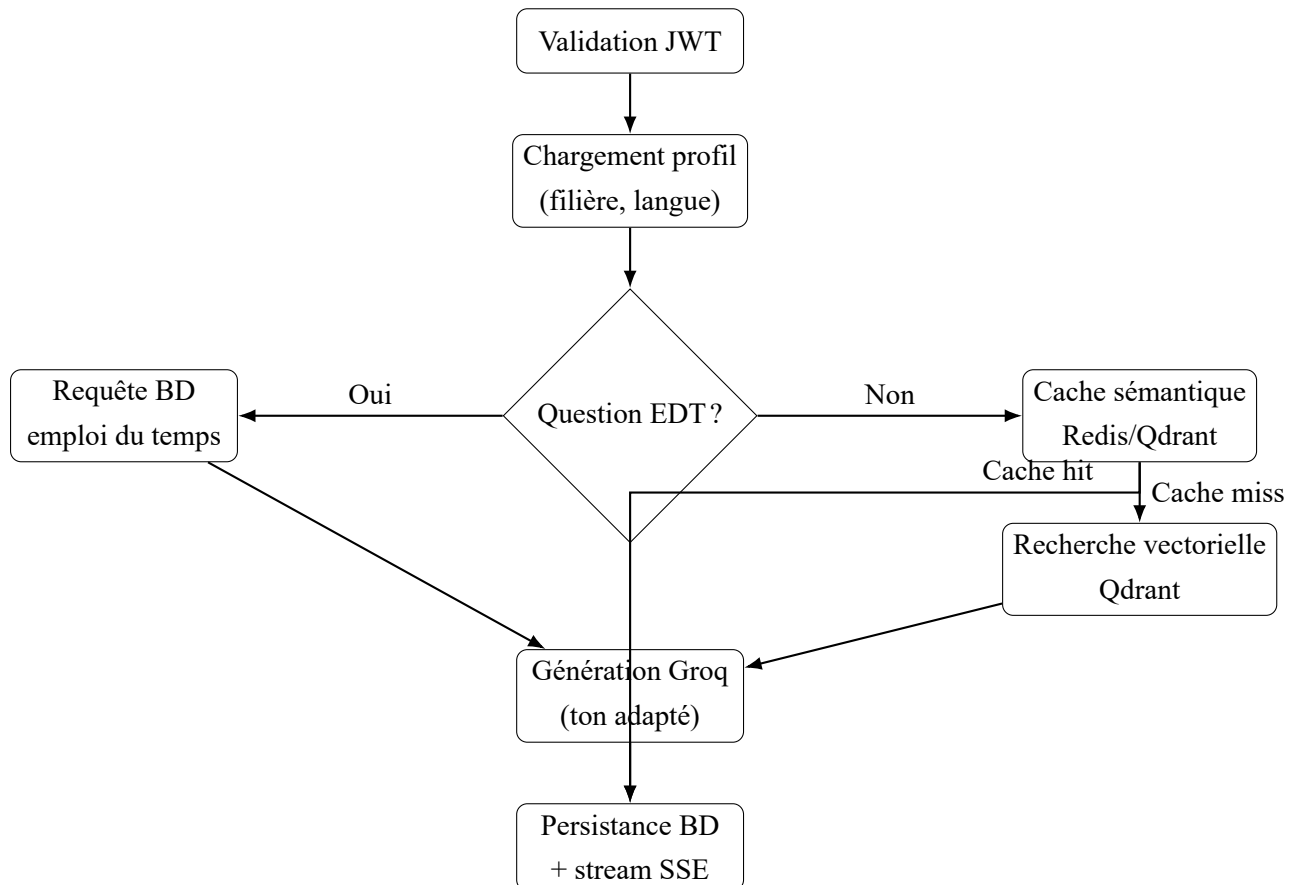


FIGURE 2.2 – Cycle de traitement d’une question

Deux chemins de réponse sont volontairement séparés. Les questions liées aux emplois du temps sont traitées par des données structurées afin d’éviter toute invention d’horaire par le modèle. Les questions documentaires passent par la recherche augmentée par génération, ce qui permet au modèle de s’appuyer sur les extraits pertinents des fichiers FAQ avant de formuler la réponse.

VI. Choix technologiques

La pile technique a été sélectionnée pour répondre aux contraintes d’une application web moderne, performante et sécurisée, tout en restant reproductible sur un environnement académique.

Couche présentation. Le frontend repose sur Next.js 15.5 avec React et TypeScript 5.8,

qui apportent le routage App Router, le typage strict et la génération optimisée. Tailwind CSS 3.4 assure un stylage rapide avec support RTL natif via les variantes `rtl`. Les formulaires utilisent React Hook Form couplé à Zod pour une validation déclarative et typée côté client. L'état global (authentification et conversations) est géré par un store Zustand singleton persisté en `localStorage`. Le basculement mode clair/sombre est assuré par next-themes sans rechargement de page.

Couche métier. Le backend est développé en Python 3.12 avec FastAPI pour ses performances asynchrones, sa documentation Swagger auto-générée et le typage strict via Pydantic. SQLAlchemy sert d'ORM avec compatibilité SQLite pour les tests isolés. Alembic 1.14 gère les migrations versionnées et les applique automatiquement au démarrage. SlowAPI protège les routes sensibles par limitation de débit (10 req/min) et Loguru assure une journalisation structurée vers fichier et console.

Couche données. PostgreSQL 15 offre la robustesse relationnelle et les transactions ACID pour la persistance principale. Le cache Redis 7 est exploité à deux niveaux (exact et sémantique) pour réduire les appels LLM coûteux. Le moteur vectoriel Qdrant 1.17 avec FastEmbed et le modèle BAAI/bge-small-en-v1.5 (384 dimensions, optimisé CPU) prend en charge la recherche sémantique locale sans GPU.

Intelligence artificielle et internationalisation. Le modèle LLaMA-3.3-70B est exposé via Groq Cloud pour sa rapidité de génération et son support natif du streaming SSE. L'internationalisation repose sur `i18next` 26 et `react-i18next` 17 avec détection automatique de la langue et support complet FR/EN/AR incluant la mise en page RTL pour l'arabe.

Infrastructure. Docker Compose orchestre cinq services (frontend, backend, PostgreSQL, Redis, Qdrant) et assure la reproductibilité complète de la stack sur tout environnement. Un pipeline GitHub Actions automatise les tests backend et le build frontend à chaque *push* sur la branche principale.

TABLE 2.4 – Récapitulatif des technologies par couche

Couche	Technologies
Présentation	Next.js 15.5, React, TypeScript 5.8, Tailwind CSS 3.4, Zustand, React Hook Form + Zod, next-themes
Métier	FastAPI (Python 3.12), SQLAlchemy, Alembic 1.14, Pydantic, SlowAPI, Loguru
Données	PostgreSQL 15, Redis 7, Qdrant 1.17, FastEmbed (BAAI/bge-small-en-v1.5, 384 dim.)
IA et internationalisation	Groq Cloud LLaMA-3.3-70B, <code>i18next</code> 26, <code>react-i18next</code> 17
Infrastructure	Docker Compose, GitHub Actions

TABLE 2.5 – Détails IA et recherche sémantique réellement implémentés

Élément	Implémentation dans le projet
Modèle d’embedding	BAAI/bge-small-en-v1.5 via FastEmbed, vecteurs de 384 dimensions et exécution locale CPU, avec repli déterministe par hachage si FastEmbed est indisponible.
Base vectorielle	Qdrant, distance cosinus, collection <code>faq_campus</code> pour la FAQ et <code>semantic_cache</code> pour le cache sémantique.
LLM	llama-3.3-70b-versatile via Groq Cloud, utilisé pour la génération, la reformulation et le streaming SSE.
Découpage documentaire	Les paires question/réponse JSON sont indexées comme unités. Les PDF sont découpés par paragraphes avec une cible d’environ 600 caractères ; aucun chevauchement fixe en tokens n’est configuré dans la version actuelle.
Récupération	Top-K final de 5 passages après recherche vectorielle élargie, pondération lexicale et filtrage des sources actives.

VII. Conclusion

L’architecture retenue est modulaire, évolutive et adaptée aussi bien à une démonstration académique qu’à une future mise en production. La séparation stricte des responsabilités entre les services facilite la maintenance, les tests indépendants et la montée en charge. Les choix technologiques ont été guidés par des critères de maturité, de performance et de compatibilité avec les contraintes du projet. Le chapitre suivant détaille la réalisation concrète de chacun de ces composants, de l’organisation du backend FastAPI jusqu’à l’interface d’administration.

Chapitre III

Réalisation et implémentation

I. Introduction

Ce chapitre décrit en détail les différentes parties développées : backend et son organisation en services spécialisés, moteur RAG, intégration LLM, gestion des emplois du temps, stratégie de cache à deux couches, sécurité et protection des données, internationalisation et interface d'administration. Pour chaque composant, les choix d'implémentation sont expliqués en relation avec les exigences définies au chapitre I et l'architecture conçue au chapitre II. Ce chapitre est le plus dense du rapport car il couvre l'intégralité de la réalisation technique, tant côté backend que frontend.

II. Backend FastAPI

1. Organisation et initialisation

Le backend expose trois groupes de routes regroupés sous le préfixe `/api/v1` : les routes d'authentification (`/auth`), de chat (`/chat`) et d'administration (`/admin`). Au démarrage, le cycle de vie de l'application (*lifespan*) initialise la base de données via `db_service.init_db()`, applique les migrations Alembic en attente et crée le compte administrateur à partir de la variable d'environnement `ADMIN_PASSWORD`. La documentation Swagger générée automatiquement sur `/docs` facilite les tests manuels, l'intégration frontend et la démonstration académique.

2. Pile de middleware

Avant d'atteindre les routes, chaque requête traverse une pile de middleware : les en-têtes de sécurité HTTP sont ajoutés systématiquement, la limitation de débit est appliquée via `SlowAPI` (10 requêtes par minute pour le chat), et la configuration `CORS` autorise explicitement les origines frontales déclarées.

TABLE 3.1 – En-têtes de sécurité HTTP appliqués à chaque réponse

En-tête	Valeur et protection
<code>X-Content-Type-Options</code>	<code>nosniff</code> – empêche le reniflage MIME.
<code>X-Frame-Options</code>	<code>DENY</code> – interdit l'encapsulation (clickjacking).
<code>Referrer-Policy</code>	<code>no-referrer</code> – évite la fuite d'URL dans les requêtes sortantes.
<code>Permissions-Policy</code>	Désactive caméra, microphone et géolocalisation.
<code>Strict-Transport-Security</code>	Activé en production (<code>max-age=31536000</code>).

3. Sondes de disponibilité

Deux endpoints de santé sont exposés : GET /healthz pour une sonde légère (toujours 200 OK), et GET / pour un contrôle complet qui teste la connectivité à Qdrant et à Groq Cloud, retournant plus de 30 champs de statut utiles lors du démarrage des conteneurs Docker.

III. Moteur RAG

1. Vue d'ensemble

Le moteur RAG (*Retrieval-Augmented Generation*) permet au chatbot de répondre aux questions générales à partir d'une base documentaire institutionnelle. Les documents sont vectorisés localement avec le modèle BAAI/bge-small-en-v1.5 (384 dimensions, via FastEmbed, optimisé CPU) et stockés dans Qdrant. Deux collections sont maintenues : faq_campus pour les documents indexés, et semantic_cache pour les réponses mises en cache.

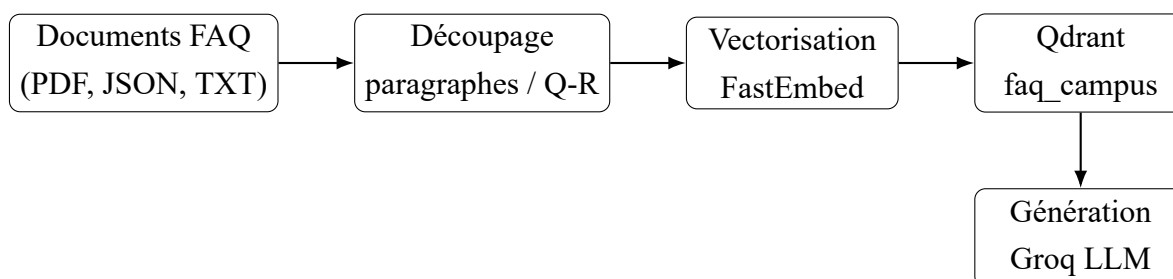


FIGURE 3.1 – Chaîne de traitement RAG

2. Sources documentaires

La base documentaire active est placée dans backend/data/faq. Elle contient notamment une base FAQ officielle structurée en JSON et sa version PDF, tandis que les anciens documents sont conservés dans une archive interne pour traçabilité. Ces sources couvrent les procédures d'inscription, les filières, les contacts administratifs et la vie étudiante. L'administrateur peut compléter ou remplacer ces fichiers à tout moment depuis le tableau de bord sans redéploiement.

3. Pipeline de récupération

Le pipeline de récupération comprend plusieurs étapes successives. La requête est d'abord prétraitée : normalisation de la casse et des accents, détection de la langue par analyse de script et règles simples, puis enrichissement par variantes synonymiques pour améliorer le rappel. L'intention est classifiée parmi plusieurs catégories (*contact*, *localisation*, *liste*, *définition*, *général*) afin de pondérer les résultats les plus pertinents. La recherche vectorielle est effectuée dans Qdrant avec une similarité cosinus, puis les passages candidats sont reclassés par un score combinant proximité vectorielle, correspondance lexicale et qualité de source. En cas d'indisponibilité de Qdrant, un mécanisme de repli lexical garantit la continuité du service.

4. Évaluation de la confiance

Chaque réponse RAG est accompagnée d'un niveau de confiance calculé à partir du score de similarité du meilleur résultat :

TABLE 3.2 – Niveaux de confiance RAG

Score de similarité	Niveau	Interprétation
$\geq 0,70$	high	Contexte proche et réponse appuyée par les passages récupérés.
$\geq 0,40$ et $< 0,70$	medium	Contexte exploitable mais partiel, réponse à interpréter avec prudence.
$< 0,40$	low	Contexte insuffisant, réponse potentiellement incomplète.

Ce score est retourné dans la réponse API et stocké dans les métadonnées JSON du message. Il indique une proximité sémantique avec les passages récupérés, mais ne constitue pas une preuve automatique de vérité.

5. Gestion des documents FAQ

L'administrateur peut importer des fichiers FAQ aux formats PDF, TXT, Markdown et JSON directement depuis le tableau de bord. Chaque import déclenche optionnellement une reconstruction automatique de l'index Qdrant. Les fichiers peuvent être prévisualisés, téléchargés ou supprimés avant toute action, ce qui permet de contrôler le contenu indexé. La reconstruction peut également être déclenchée manuellement via l'endpoint `POST /api/v1/admin/faq/rebuild` ou le script `rebuild_faq_index.py`.

IV. Intégration LLM – Groq Cloud

1. Usages du modèle

Le modèle `llama-3.3-70b-versatile` est utilisé pour cinq tâches distinctes :

1. la génération de réponses FAQ à partir du contexte RAG récupéré, avec les cinq derniers messages de la conversation comme historique ;
2. la reformulation en langage naturel des données d'emploi du temps structurées extraites de PostgreSQL ;
3. la génération automatique des titres de conversations (repli sur les 50 premiers caractères de la question si l'appel échoue) ;
4. la détection de langue et la traduction entre le français, l'anglais et l'arabe pour assurer les réponses dans la langue de l'utilisateur ;
5. le raffinement de la détection des émotions lorsque les règles heuristiques sont insuffisantes.

Tous les appels disposent d'un délai d'expiration de 90 secondes et d'une gestion d'erreur gracieuse retournant un message d'excuse en cas d'échec.

2. Masquage des données personnelles

Avant chaque appel au LLM, les données personnelles identifiables (PII) sont automatiquement masquées : les adresses email sont remplacées par [EMAIL] et les numéros de téléphone par [PHONE]. Ce traitement réduit l'exposition de données personnelles vers l'infrastructure cloud de Groq et s'inscrit dans le principe de minimisation des données.

3. Protection contre l'injection de prompt

Les requêtes utilisateur sont analysées avant soumission. Les tentatives d'injection détectées par des motifs prédéfinis (instructions d'ignorer le contexte, changement de rôle du modèle, etc.) sont neutralisées. La séparation stricte entre le rôle système et le contenu utilisateur dans la structure des messages empêche toute élévation de privilège via le LLM. Les mots vides (*stop-words*) sont également filtrés des requêtes FAQ avant vectorisation afin d'améliorer la pertinence des embeddings.

V. Routage et détection des émotions

1. Routage des questions

Le service de routage analyse chaque question pour déterminer si elle porte sur un emploi du temps ou sur la FAQ générale. La détection repose sur une liste de mots-clés heuristiques (jours de la semaine, « cours », « edt », « schedule », etc.) complétée par une résolution floue du code de filière pour identifier la classe de l'étudiant. La résolution floue applique trois niveaux de correspondance : exacte, préfixe, puis contenance dans la chaîne de caractères. La liste des classes disponibles est mise en cache et rafraîchie à chaque modification des emplois du temps publiés.

2. Détection des émotions

Le module de détection des émotions utilise une approche hybride. Une première passe basée sur des listes de marqueurs multilingues (français, anglais, arabe) identifie l'état émotionnel avec gestion de la négation. Si la détection est inconcluante, un appel LLM léger confirme ou affine le résultat.

TABLE 3.3 – États émotionnels détectés et adaptation du ton

État	Adaptation de la réponse
urgent	Ton direct, information essentielle en priorité.
stressed	Ton rassurant, réponse structurée étape par étape.
anxious	Explication claire, éléments de réassurance.
frustrated	Ouverture empathique, solution directe et actionnable.
sad	Ton encourageant, clôture motivationnelle.
neutral	Réponse informative standard.

VI. Stratégie de cache à deux couches

Le projet implémente un cache à deux niveaux pour réduire les appels LLM coûteux et améliorer la latence des réponses fréquentes.

La première couche (L1) est un cache exact dans Redis : la clé est un haché SHA-256 des 16 premiers caractères de la question normalisée, formatée comme chatbot : `{prefix}:{sha256[:16]}`. La deuxième couche (L2) est un cache sémantique combinant Redis et Qdrant : si la correspondance exacte échoue, une recherche de similarité vectorielle dans la collection `semantic_cache` détecte les questions paraphrasées. En cas de succès, la réponse mise en cache est retournée en streaming par blocs de trois mots pour maintenir l’expérience utilisateur.

TABLE 3.4 – Conditions de contournement du cache

Condition	Justification
Question portant sur un emploi du temps	La réponse dépend de la date courante et varie jour après jour.
Conversation multi-tours avec historique	Le contexte des messages précédents rend chaque réponse unique.
Réponse d’erreur ou d’échec LLM	Les erreurs ne doivent pas être propagées aux utilisateurs suivants.

Le TTL par défaut est de 3 600 secondes (1 heure). L’administrateur peut vider intégralement le cache Redis depuis le tableau de bord via `POST /api/v1/admin/cache/flush`. Le cache est invalidé automatiquement après chaque reconstruction de l’index vectoriel. Les statistiques sur les questions les plus fréquentes sont extraites des correspondances cache via la fonction `get_most_asked_questions()`.

VII. Gestion des emplois du temps

Les emplois du temps sont stockés dans PostgreSQL sous forme de données structurées avec classe, année universitaire, semestre et créneaux. Chaque créneau comprend le jour de la semaine (0 = lundi), l’heure de début et de fin, la matière, le professeur, la salle et le type de

séance (Cours, TD ou TP). Le chatbot utilise un routeur métier pour identifier les questions portant sur un jour, un professeur, un module ou un planning complet. La résolution de la date utilise `dateparser` avec le fuseau horaire Africa/Casablanca, permettant des requêtes en langage naturel (« aujourd’hui », « demain », « lundi prochain »). Cette approche évite de générer librement des horaires et garantit que la réponse provient exclusivement de la base de données.

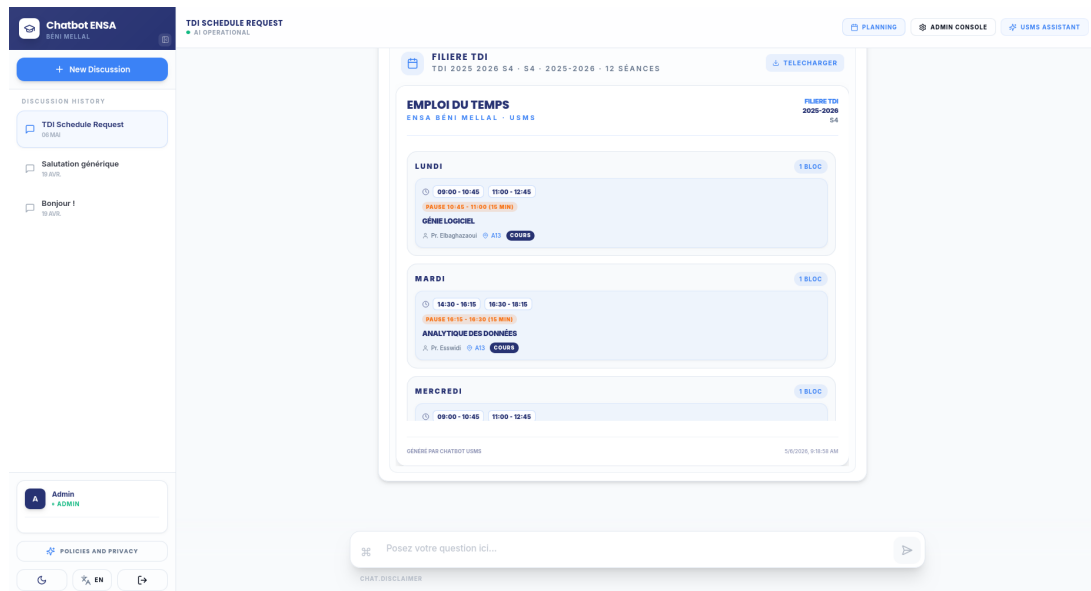


FIGURE 3.2 – Affichage structuré d’un emploi du temps dans le chatbot

TABLE 3.5 – Séparation entre LLM seul, RAG et données structurées

Besoin	Solution utilisée	Justification
FAQ et procédures	RAG + Qdrant + LLM	Réponses naturelles ancrées sur les documents institutionnels récupérés.
Emplois du temps	PostgreSQL + routeur métier	Horaires, salles et enseignants proviennent de données structurées et non d’une génération libre.
Reformulation	LLM	Améliore la lisibilité sans devenir la source de vérité.
Questions répétitives	Redis + cache sémantique	Réduit la latence et les appels externes lorsque la question est déjà connue ou très proche.
Données utilisateurs	PostgreSQL	Garantit persistance, cohérence relationnelle et contrôle d’accès.

VIII. Frontend Next.js

1. Architecture frontend

Le frontend utilise l'App Router de Next.js 15 avec TypeScript 5.8 et Tailwind CSS 3.4, avec une séparation nette entre les composants serveur et client. L'état d'authentification est géré par un store Zustand singleton, qui persiste le jeton d'accès dans `localStorage` et expose les méthodes de connexion, déconnexion et rafraîchissement de session. Les appels API utilisent Axios avec un intercepteur qui attache automatiquement le jeton Bearer et gère le rafraîchissement transparent en cas d'expiration ; en cas d'échec du rafraîchissement, l'utilisateur est redirigé vers `/login`. Les formulaires utilisent React Hook Form couplé à Zod pour une validation côté client typée et déclarative.

L'interface est découpée en pages claires : `/login` pour la connexion et l'inscription, `/chat` pour l'assistance conversationnelle, `/admin` pour le pilotage de la plateforme, `/profile` pour les préférences et les droits de confidentialité, et `/privacy` pour l'information de transparence. Le middleware Next.js protège ces routes en lisant le jeton et le rôle stockés dans les cookies, et redirige les accès non autorisés. Le déploiement utilise une construction Docker multi-étapes avec sortie *standalone* pour minimiser la taille de l'image finale.

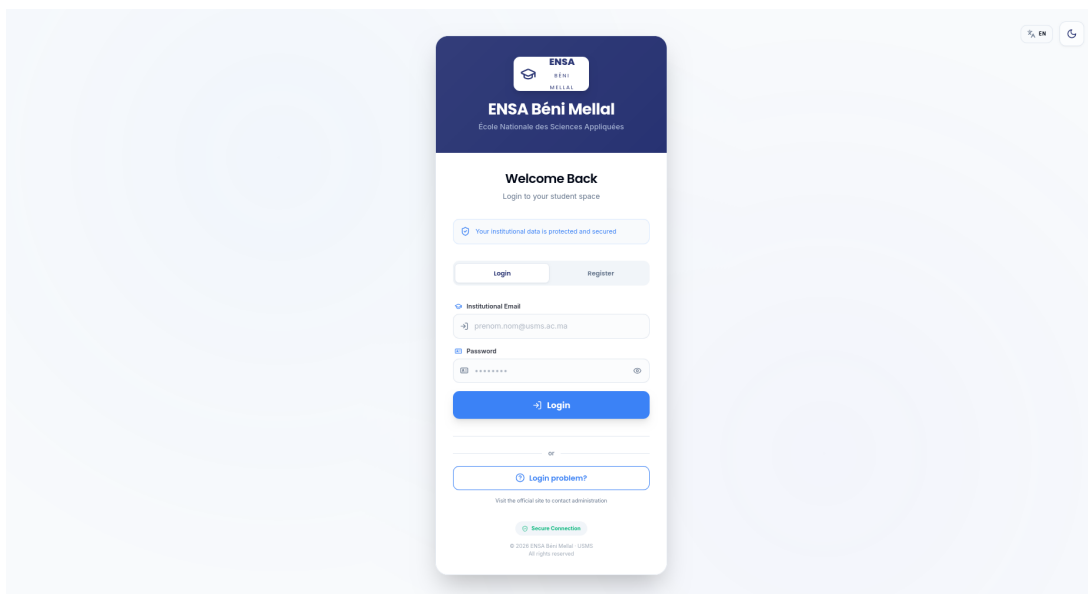


FIGURE 3.3 – Page de connexion sécurisée de l'application

2. Streaming et interface de chat

Les réponses du chatbot sont reçues en streaming via Server-Sent Events (SSE). Le frontend lit le flux de données par segments et les affiche progressivement, simulant un effet de frappe. Les métadonnées de fin de flux (fichier source, confiance RAG, données d'emploi du temps structurées) sont transmises sous la forme de chunks préfixés `__metadata__` : et traitées séparément, affichées sous la réponse textuelle.

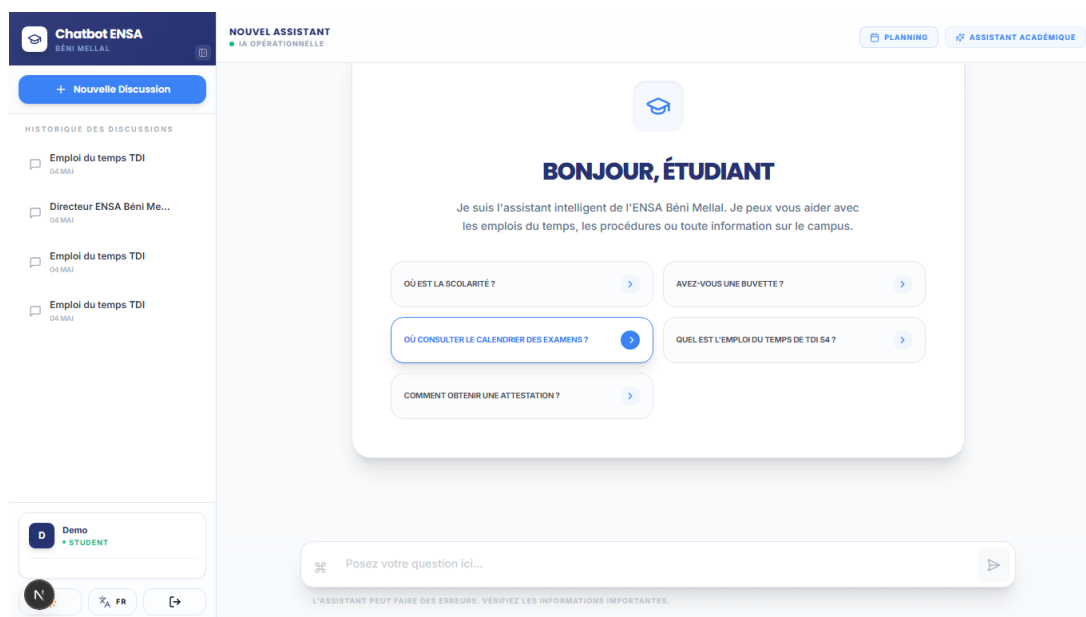


FIGURE 3.4 – Interface principale du chatbot côté étudiant

Lorsqu'une question porte sur les documents institutionnels, le chatbot génère une réponse contextualisée à partir du moteur RAG et affiche la source documentaire ainsi que le niveau de confiance associé.

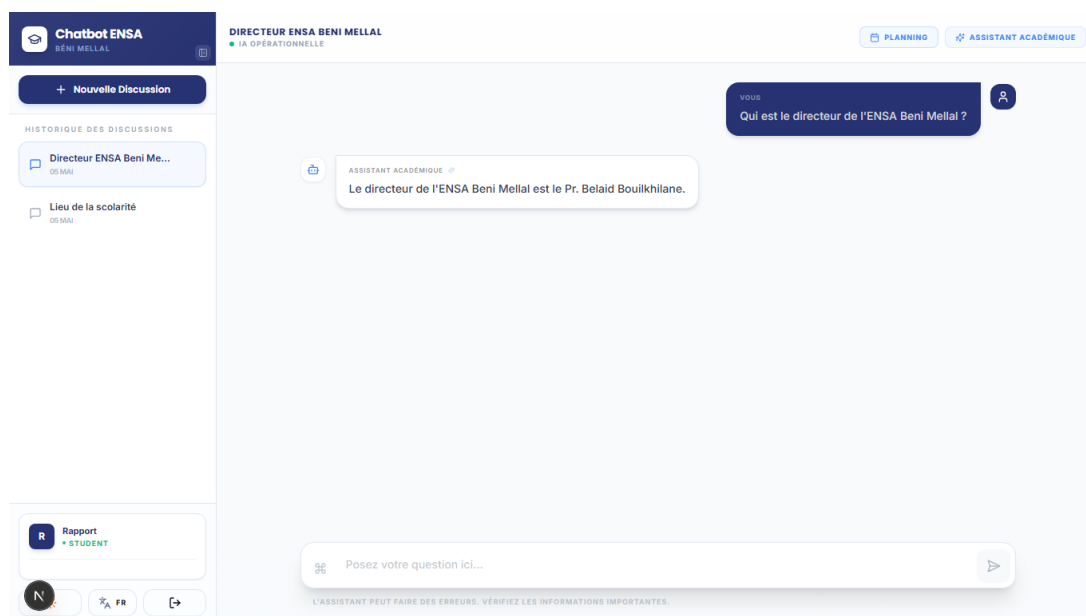


FIGURE 3.5 – Exemple de réponse issue de la base de connaissances FAQ

3. Internationalisation

L'interface est entièrement traduite en français, anglais et arabe via i18next 26 et react-i18next 17. Les fichiers de traduction (fr.json, en.json, ar.json) partagent une hiérarchie de clés commune couvrant l'authentification, le chat, l'administration, le profil et la confidentialité. La détection de langue s'effectue dans l'ordre : préférence stockée en base, préférence localStorage, langue du navigateur, puis français par défaut. Lorsque l'arabe est actif, l'attribut dir="rtl" est appliqué à la racine du document et les variantes Tailwind rtl: ajustent la

mise en page. La langue préférée est synchronisée avec le serveur via `POST /api/v1/auth/me/language`.

Plusieurs composants spécialisés contribuent à l'expérience utilisateur : MessageBubble pour l'affichage des réponses Markdown et des badges de confiance, TimetableGrid pour la représentation visuelle des emplois du temps, ChatInput pour la saisie avec raccourcis clavier, et EdtForm côté administrateur pour valider les imports JSON avant publication. Le basculement clair/sombre est géré par next-themes sans rechargement de page.

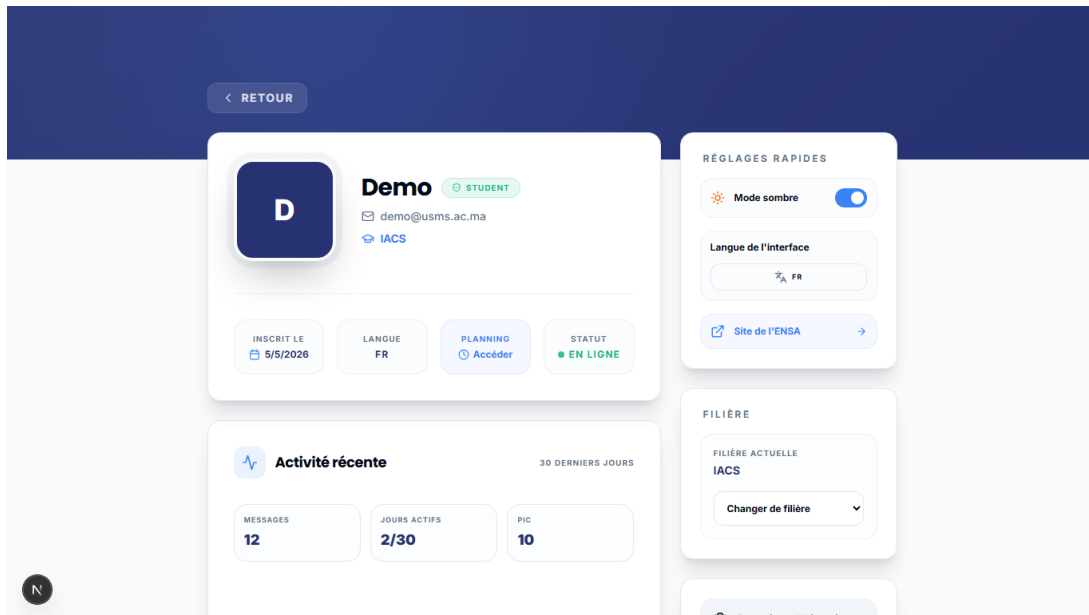


FIGURE 3.6 – Page profil étudiant

Le module de planification des révisions, accessible depuis le profil, illustre l'adaptabilité de l'interface aux besoins personnalisés de l'étudiant, indépendamment de la langue active.

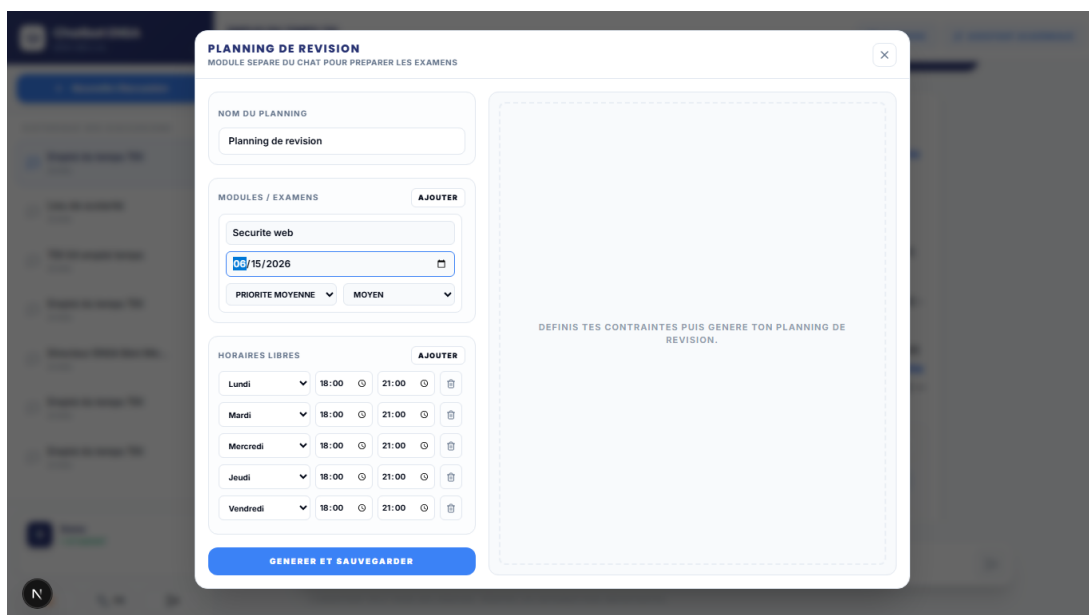


FIGURE 3.7 – Module de planification des révisions

IX. Axe cybersécurité

Étant réalisé dans le cadre de la filière IACS, le projet ne se limite pas à l'intégration d'un modèle d'intelligence artificielle. Il traite aussi la protection d'une application web universitaire manipulant des comptes, des rôles, des conversations, des documents FAQ et des opérations administrateur.

1. Objectifs de sécurité

L'analyse s'appuie sur les trois objectifs classiques : confidentialité, intégrité et disponibilité. La confidentialité concerne les comptes utilisateurs, les conversations et les secrets techniques. L'intégrité concerne les emplois du temps, les fichiers FAQ, les rôles et les réponses construites à partir des données internes. La disponibilité concerne l'accès continu au chatbot, au tableau de bord administrateur et aux services PostgreSQL, Redis et Qdrant.

TABLE 3.6 – Objectifs de sécurité du projet

Objectif	Application dans Chatbot USMS
Confidentialité	Protéger les sessions, les conversations, les comptes et les clés d'API utilisées par le backend.
Intégrité	Empêcher la modification non autorisée des utilisateurs, des FAQ, des emplois du temps et des demandes de filière.
Disponibilité	Limiter les abus de l'API, maintenir les services de données et prévoir une stack reproductible avec Docker.
Traçabilité	Conserver les messages, les feedbacks, les questions faibles et les actions de gestion nécessaires au suivi.

2. Authentification et gestion des sessions

L'inscription impose une adresse institutionnelle (@usms.ac.ma ou @usms.ma) afin de limiter l'accès au périmètre universitaire. Les mots de passe sont hachés avec bcrypt (facteur de coût 12), ce qui évite le stockage de mots de passe en clair et rend les attaques par dictionnaire computationnellement coûteuses.

Après connexion, le backend délivre deux jetons : un jeton d'accès JWT à courte durée de vie (15 minutes) et un jeton de rafraîchissement à longue durée (7 jours). Ce dernier est stocké dans un cookie HttpOnly, SameSite=Strict, inaccessible au JavaScript côté client, ce qui protège contre le vol de session par des scripts malveillants et les attaques CSRF. Le jeton d'accès est renouvelé automatiquement par l'intercepteur Axios du frontend avant chaque expiration ; le jeton et le rôle sont également stockés dans des cookies accessibles au middleware Next.js pour le filtrage des routes protégées.

TABLE 3.7 – Paramètres de configuration des jetons JWT

Paramètre	Valeur par défaut	Variable d’environnement
Algorithme	HS256	—
Durée du jeton d’accès	15 minutes	ACCESS_TOKEN_EXPIRE_MINUTES
Durée du jeton de rafraîchissement	7 jours	REFRESH_TOKEN_EXPIRE_DAYS
Cookie refresh	HttpOnly, Same-Site=Strict, Secure (HTTPS)	—
Clé secrète	(obligatoire, ≥ 32 caractères)	JWT_SECRET_KEY

Si la clé `JWT_SECRET_KEY` est absente ou trop faible, le backend génère automatiquement une clé éphémère et émet un avertissement au démarrage. Ce mécanisme empêche le démarrage silencieux avec un secret par défaut, au prix d’une invalidation des sessions lors d’un redémarrage du serveur.

3. Protection contre le force brute

Le système de verrouillage de compte protège contre les attaques par force brute. Après cinq tentatives de connexion échouées consécutives, le compte est verrouillé pendant 30 minutes. Le compteur est remis à zéro dès une connexion réussie. L’heure de déverrouillage est communiquée à l’utilisateur dans le message d’erreur.

4. Politique de mots de passe

Les mots de passe doivent comporter au moins 8 caractères, un chiffre et un caractère spécial. Les anciens mots de passe hachés en SHA-256 (format hérité) sont automatiquement migrés vers `bcrypt` lors de la prochaine connexion réussie.

5. Contrôle d’accès basé sur les rôles (RBAC)

La séparation des rôles est vérifiée côté backend pour chaque opération. Un étudiant peut accéder à ses propres données; un administrateur peut gérer les données globales de la plateforme. Les routes admin sont protégées par la dépendance `FastAPI require_admin()` qui lève une erreur HTTP 403 si le rôle n’est pas admin. La propriété des ressources est vérifiée pour chaque conversation et chaque message, afin d’empêcher les accès non autorisés entre comptes (protection IDOR).

TABLE 3.8 – Menaces principales et mesures appliquées

Menace	Mesure appliquée
Accès non autorisé	Contrôle RBAC, séparation étudiant/admin et vérification côté backend des routes sensibles.
Usurpation de session	JWT signés HMAC-SHA256, cookies HttpOnly + SameSite=Strict, expiration courte et protection CSRF.
Force brute	Verrouillage de compte après 5 échecs consécutifs pendant 30 minutes.
Comptes non institutionnels	Restriction des inscriptions aux emails @usms.ac.ma et @usms.ma.
Injection ou entrée invalide	Validation des schémas Pydantic, typage des données et contrôle des paramètres reçus.
Fichier FAQ dangereux	Validation des extensions autorisées, isolation dans le dossier FAQ et extraction contrôlée.
Injection de prompt LLM	Détection de motifs d'injection, isolation du rôle système et masquage des PII.
Abus de ressources	Limitation de débit (10 req/min), cache Redis et séparation des traitements coûteux.
Exposition de secrets	Variables d'environnement exclusivement, aucune clé sensible dans le code source.

6. Protection des données

Le projet applique plusieurs principes inspirés du Règlement Général sur la Protection des Données, sans prétendre constituer une certification juridique complète. Le droit à l'effacement est représenté par l'endpoint DELETE /api/v1/auth/me, qui supprime le compte et les données associées en cascade. La portabilité est couverte par GET /api/v1/auth/me/export, qui retourne un fichier JSON structuré contenant le profil et l'historique complet. La minimisation est renforcée par le masquage automatique des PII avant chaque appel au LLM cloud, et la responsabilité par la journalisation des actions sensibles dans la table audit_logs. Pour un déploiement réel au Maroc, cette approche devrait être complétée par une analyse de conformité à la loi 09-08 relative à la protection des personnes physiques à l'égard du traitement des données à caractère personnel, ainsi que par les exigences de la CNDP.

7. Limites et durcissement recommandé

Les mesures intégrées sont adaptées au périmètre d'un PFA. Une exploitation institutionnelle demanderait un durcissement supplémentaire : HTTPS obligatoire avec HSTS, vérification par email à l'inscription, sauvegardes planifiées, supervision applicative, rotation des secrets, authentification double facteur pour l'administrateur, scan régulier des dépendances et tests de

pénétration OWASP ciblés.

X. Administration

L'espace administrateur constitue le centre de contrôle de la plateforme. L'administrateur peut consulter les statistiques (nombre d'utilisateurs, de conversations, de messages, questions les plus fréquentes, score de feedback), gérer les utilisateurs, importer et supprimer des fichiers FAQ, reconstruire l'index Qdrant, vider le cache Redis, publier les emplois du temps et traiter les demandes de changement de filière.

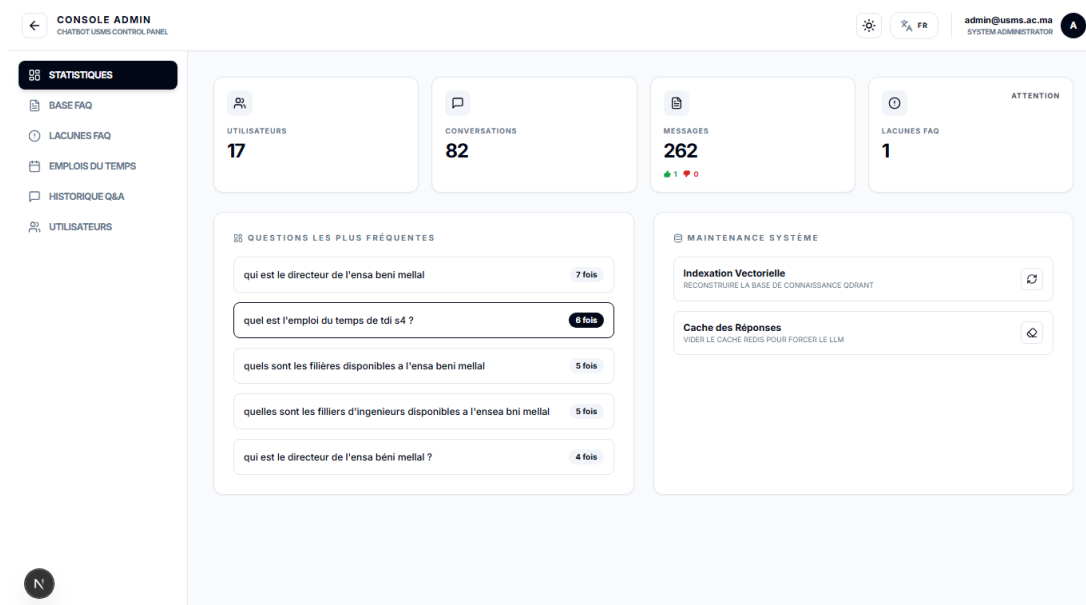


FIGURE 3.8 – Tableau de bord administrateur

Une attention particulière a été portée aux actions de visualisation avant décision. Pour les fichiers FAQ, l'administrateur peut ouvrir un aperçu du document et le télécharger avant suppression ou réindexation. Pour les emplois du temps publiés, il peut consulter le détail d'un planning, le dupliquer dans le formulaire comme modèle, puis le modifier avant publication. Les questions ayant reçu des retours négatifs sont automatiquement enregistrées dans la table `qa_review` et accessibles depuis le tableau de bord pour amélioration.

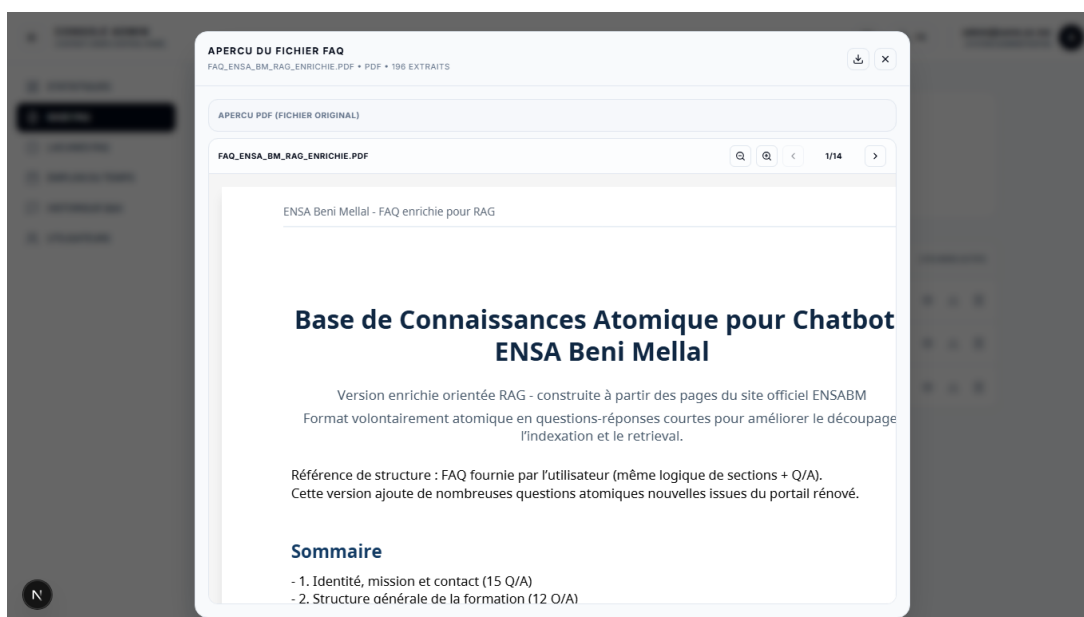


FIGURE 3.9 – Aperçu d’un document FAQ avant action administrateur

TABLE 3.9 – Fonctions disponibles dans le tableau de bord administrateur

Bloc fonctionnel	Capacités offertes
Statistiques	Nombre d’utilisateurs, conversations, messages, questions fréquentes et questions faibles à améliorer.
Base FAQ	Upload (PDF/TXT/MD/JSON), prévisualisation, téléchargement, suppression et reconstruction de l’index vectoriel.
Emplois du temps	Import JSON, validation via EdtForm, publication, visualisation, duplication et suppression des plannings actifs.
Utilisateurs	Recherche, consultation des profils et suivi des demandes de changement de filière.
Maintenance	Vidage du cache, nettoyage des données de plus de 180 jours et supervision du contenu de démonstration.

L’interface de gestion de la base FAQ permet à l’administrateur d’importer de nouveaux documents, de prévisualiser leur contenu et de déclencher la reconstruction de l’index vectoriel Qdrant.

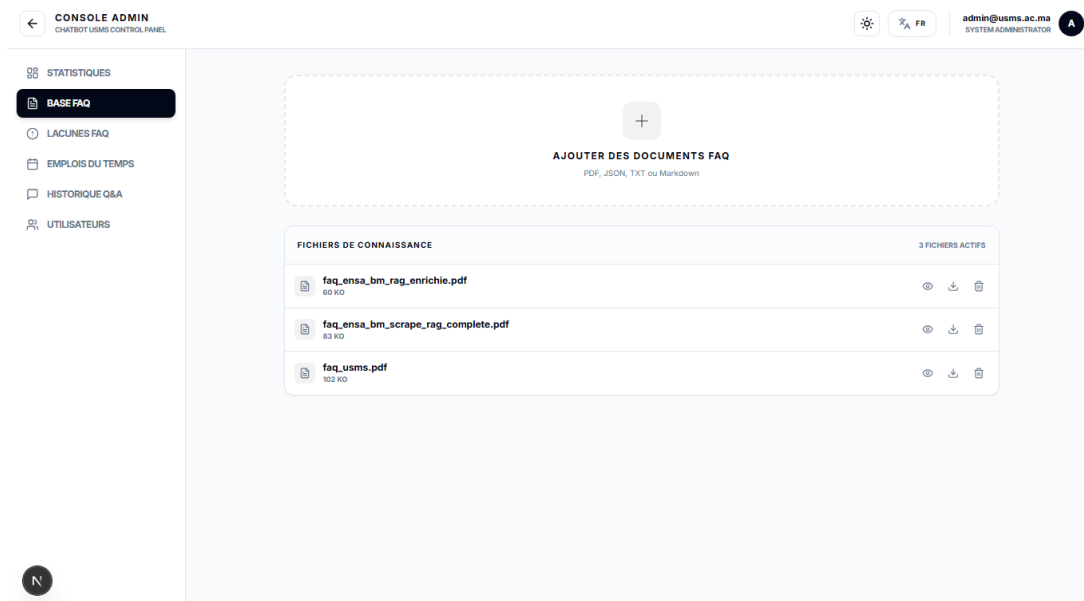


FIGURE 3.10 – Gestion de la base FAQ dans l’espace administrateur

La gestion des emplois du temps publiés offre une vue d’ensemble des plannings actifs par filière et semestre, avec la possibilité de dupliquer un planning existant comme modèle avant publication.

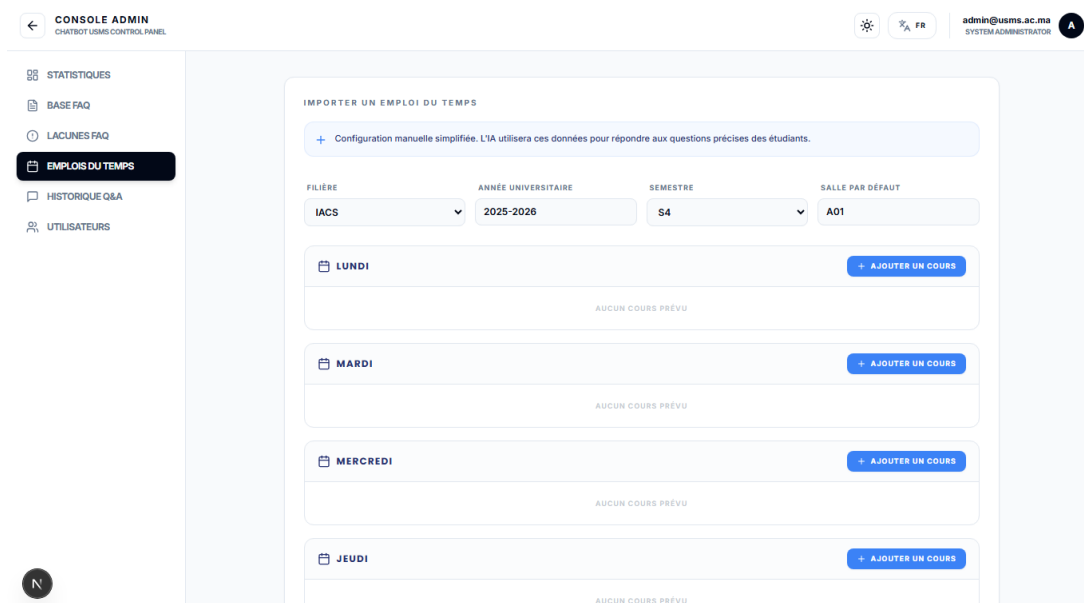


FIGURE 3.11 – Gestion des emplois du temps publiés

Enfin, la vue de gestion des utilisateurs centralise les profils étudiants et le traitement des demandes de changement de filière soumises depuis l’espace profil.

UTILISATEUR	RÔLE	FILIÈRE	CONVS	MSGS
admin@usms.ac.ma	ADMIN	IACS_2025-2026_S4	26	120
kossi_richard.allado@usms.ac.ma	STUDENT	IACS	6	40
qa_pdf_campaign_24@usms.ac.ma	STUDENT	-	10	20
qa_pdf_grounded_3@usms.ac.ma	STUDENT	-	7	14
qa_real_questions@usms.ac.ma	STUDENT	-	6	12
qa_pdf_grounded@usms.ac.ma	STUDENT	-	6	12
qa_pdf_grounded_2@usms.ac.ma	STUDENT	-	6	12
demo@usms.ac.ma	STUDENT	IACS_2025-2026_S4	5	12
factual.5d8cee06@usms.ac.ma	STUDENT	IACS	5	10
e2e.28cdb374@usms.ac.ma	STUDENT	TDI	1	4
ui.1776024907526@usms.ac.ma	STUDENT	IACS	1	2
ui.1776024951362@usms.ac.ma	STUDENT	IACS	1	2

FIGURE 3.12 – Gestion des utilisateurs et des demandes de filière

XI. Migration PostgreSQL

Le projet a évolué d’une version initiale SQLite vers PostgreSQL. Cette migration apporte une base plus robuste, des migrations Alembic versionnées, des scripts d’import d’emplois du temps et une meilleure séparation entre données partagées et données locales. Les migrations sont appliquées automatiquement au démarrage du backend via le cycle de vie de l’application, ce qui simplifie le déploiement dans les environnements Docker. La compatibilité SQLite est préservée pour la suite de tests automatisés, qui s’exécutent sur une base isolée en mémoire.

XII. Outillage

Les scripts `dev.ps1` (Windows) et `dev.sh` (Linux) préparent l’environnement complet : création du `virtualenv` Python, installation des dépendances, démarrage de PostgreSQL, Redis et Qdrant via Docker Compose, initialisation de la base, import des emplois du temps de démonstration et lancement simultané du backend et du frontend. Un script `bootstrap_dev_content.py` centralise l’initialisation des données de développement (base FAQ, emplois du temps, compte administrateur).

TABLE 3.10 – Scripts utilitaires du projet

Script	Rôle
<code>bootstrap_dev_content.py</code>	Initialisation complète : base de données, admin, emplois du temps et index FAQ.
<code>seed_timetables.py</code>	Import des emplois du temps depuis un fichier JSON (<code>--replace</code> pour réinitialiser).
<code>export_timetables_seed.py</code>	Export de la base de données vers un fichier JSON réutilisable comme graine.
<code>rebuild_faq_index.py</code>	Reconstruction de l’index vectoriel Qdrant à partir des fichiers FAQ.
<code>rag_pdf_benchmark.py</code>	Évaluation de la qualité de récupération RAG sur un jeu de requêtes de référence.

XIII. Conclusion

La réalisation aboutit à une application complète, maintenable et démontrable. L’interface étudiant est multilingue et accessible, le moteur RAG est configurable par l’administrateur sans intervention technique, l’espace d’administration couvre l’ensemble du cycle de vie du contenu, et la couverture de sécurité est adaptée au contexte universitaire. L’ensemble des composants développés respecte la séparation des responsabilités définie en conception. Le chapitre suivant présente la stratégie de tests mise en place pour valider ces réalisations, ainsi que les procédures de déploiement local et Docker.

Chapitre IV

Tests, déploiement et évaluation

I. Introduction

La validation du projet repose sur plusieurs niveaux de vérification complémentaires : une suite de tests automatisés couvrant les routes critiques du backend, des scénarios fonctionnels exécutés manuellement sur l’interface, et une vérification de la reproductibilité de la stack en environnement local et Docker. Ce chapitre présente également l’évaluation globale de la solution au regard des objectifs définis au chapitre I, ainsi que les limites identifiées et les pistes de durcissement pour une mise en production institutionnelle.

II. Tests automatisés

Les tests automatisés ont été répartis en plusieurs modules couvrant les fonctionnalités critiques du backend. La suite est exécutée via `pytest` avec une base SQLite isolée en mémoire, ce qui garantit la reproductibilité et l’indépendance vis-à-vis de l’environnement de développement. Un fichier `conf_test.py` fournit les fixtures communes : client de test FastAPI, en-têtes d’authentification et base de données de test. La documentation technique du projet annonce 127 tests backend validés sur la branche principale.

TABLE 4.1 – Modules de tests et couverture

Module	Scénarios couverts
<code>test_auth.py</code>	Connexion, inscription, JWT, rafraîchissement, verrouillage de compte, validation email et consentement et confidentialité.
<code>test_chat.py</code>	Envoi de question, streaming SSE, création de conversation, feedback et contrôle de propriété.
<code>test_admin.py</code>	Statistiques, import FAQ, reconstruction Qdrant, vidage cache, publication et suppression des emplois du temps.
<code>test_rag_quality.py</code>	Qualité de récupération, repli lexical, cache sémantique et gestion multilingue.
<code>test_sentiment_detection.py</code>	Détection émotionnelle FR/EN/AR, négation et scores de confiance.
<code>test_timetable_seed.py</code>	Import des graines JSON, récupération par filière et résolution floue des labels.
<code>test_db_improvements.py</code>	CRUD, suppressions en cascade, migration des mots de passe et journalisation d’audit.

Module	Scénarios couverts
test_health.py test_utils.py	et Sondes de disponibilité, fonctions utilitaires, filtrage des créneaux et masquage des PII.

III. Pipeline d'intégration continue

Un pipeline GitHub Actions automatise la validation à chaque *push* et *pull request* sur la branche principale. Il exécute deux tâches indépendantes : la suite pytest backend et le build Next.js frontend. La tâche backend installe les dépendances Python, configure les variables d'environnement de test, puis lance pytest avec la variable PYTHONPATH=. depuis la racine du projet. La tâche frontend installe les dépendances Node.js et exécute npm run build pour vérifier la compilation TypeScript et la génération de l'application. Une troisième étape optionnelle construit les images Docker pour valider la reproductibilité des Dockerfiles. Ce pipeline garantit la non-régression à chaque évolution du code source.

IV. Scénarios de validation fonctionnelle

Les principaux scénarios validés manuellement sont : inscription étudiant avec acceptation de la politique de confidentialité, connexion, question FAQ en français et en anglais, question sur l'emploi du temps avec résolution de date (« aujourd'hui », « lundi »), consultation et suppression de l'historique, feedback positif et négatif, connexion administrateur, import et aperçu d'un fichier FAQ, reconstruction de l'index, publication d'un emploi du temps, traitement d'une demande de changement de filière, export des données personnelles et suppression de compte.

V. Déploiement local

Listing 4.1 – Démarrage sous Windows

```
1 .\dev.ps1
```

Listing 4.2 – Démarrage sous Linux

```
1 bash ./dev.sh
```

Ces scripts automatisent la préparation complète de l'environnement : configuration du fichier .env, création du virtualenv Python, installation des dépendances, démarrage des services PostgreSQL/Redis/Qdrant, initialisation de la base et lancement coordonné du backend et du frontend.

VI. Déploiement Docker

Listing 4.3 – Démarrage de la stack Docker complète

```
1 | docker compose up -d --build
```

Les services sont exposés sur `localhost:3000` pour le frontend, `localhost:8000` pour l'API, `localhost:6333` pour Qdrant et `localhost:5432` pour PostgreSQL. Redis complète la pile sur le port 6379. Les données PostgreSQL et Qdrant sont persistées dans des volumes Docker nommés (`postgres_data` et `qdrant_data`). La stack est reproductible sur tout environnement disposant de Docker Compose.

Le déploiement conteneurisé repose sur cinq services : frontend, backend, postgres, redis et qdrant. Le frontend est construit en multi-étapes avec sortie *standalone* pour réduire la taille de l'image. Le backend utilise `python:3.10-slim` avec `uvicorn`. Cette organisation reproduit fidèlement l'architecture réelle du projet et facilite les démonstrations, les tests d'intégration et la reprise de l'environnement sur une autre machine.

TABLE 4.2 – Variables d'environnement requises pour le déploiement

Variable	Description
GROQ_API_KEY	Clé d'API GroqCloud pour l'accès au LLM.
JWT_SECRET_KEY	Secret de signature des jetons JWT (≥ 32 caractères aléatoires).
DATABASE_URL	Chaîne de connexion PostgreSQL pour le développement local.
DOCKER_DATABASE_URL	URL PostgreSQL pour Docker (hôte postgres).
ADMIN_PASSWORD	Mot de passe du compte administrateur initial.
POSTGRES_USER	Utilisateur de la base PostgreSQL (Docker).
POSTGRES_PASSWORD	Mot de passe PostgreSQL (Docker).
POSTGRES_DB	Nom de la base PostgreSQL (Docker).
ENABLE_CACHE	Active/désactive le cache Redis (<code>true/false</code>).
CORS_ORIGINS	Origines autorisées pour les requêtes cross-origin.

Chatbot USMS — API 2.0.0 OAS 3.1

/openapi.json

API REST du chatbot intelligent ENSAM Béni Mellal.

Fonctionnalités : FAQ dynamique, emplois du temps, calendrier des examens, dashboard administrateur.

LLM : GroqCloud LLaMA-3.3-70B | Vecteurs : Qdrant local | Auth : JWT

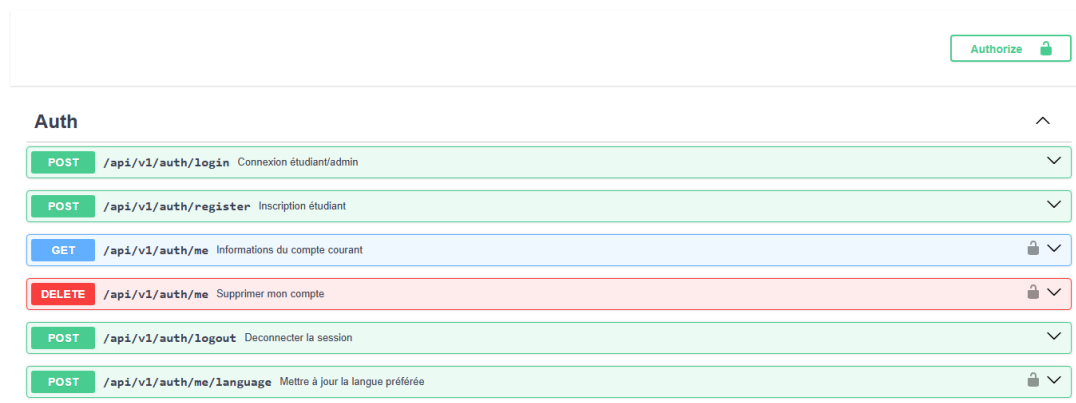


FIGURE 4.1 – Documentation Swagger de l’API FastAPI

VII. Évaluation

La solution répond aux objectifs initiaux : un point d’entrée unique pour l’étudiant, des réponses FAQ fondées sur des documents avec score de confiance, des emplois du temps issus de données structurées, une détection des émotions pour personnaliser les réponses, une administration complète du contenu, une prise en compte sérieuse de la cybersécurité applicative et de la protection des données, et une stack reproductible.

TABLE 4.3 – Bilan d’évaluation par objectif

Objectif	Niveau d’atteinte
Accès centralisé à l’information	Atteint : le chatbot réunit FAQ, emplois du temps, profils et fonctions administratives dans une même application.
Réponses fiables	Atteint sous contrôle : les FAQ reposent sur un contexte RAG et les emplois du temps sur des données structurées, mais la qualité dépend du corpus importé.
Sécurité applicative	Atteint : JWT courts, cookies HttpOnly+SameSite=Strict, protection CSRF, RBAC, audit et verrouillage de compte sont intégrés.
Protection des données	Mis en place : consentement, export, suppression, minimisation des données et journalisation sont implémentés ; la conformité juridique complète reste à valider avant production.
Exploitabilité technique	Atteint : scripts locaux, Docker Compose, Swagger et pipeline CI facilitent l’usage et l’évolution du projet.

Protocole d'évaluation IA proposé

La version actuelle valide le comportement du RAG par des tests automatisés et des scénarios fonctionnels, mais ne produit pas encore de scores RAGAS officiels. Pour renforcer l'évaluation scientifique, un protocole futur peut s'appuyer sur un jeu de 30 à 50 questions représentatives de la FAQ, des emplois du temps, des questions multilingues et des cas hors corpus. Les métriques proposées sont la *faithfulness*, l'*answer relevancy*, la *context precision*, la *context recall*, ainsi que Recall@K et MRR pour mesurer directement la qualité de récupération Qdrant.

Les limites identifiées restent maîtrisées. La pertinence des réponses dépend de la qualité des documents FAQ importés. L'appel au modèle Groq impose une connexion externe et une clé API valide. Une mise en production institutionnelle nécessiterait une configuration serveur complète avec HTTPS, une supervision applicative, une politique de sauvegarde, une rotation des secrets, une vérification email à l'inscription et un audit de sécurité.

VIII. Conclusion

Les tests automatisés et les scénarios fonctionnels montrent que le projet est prêt pour une démonstration de soutenance. Le pipeline CI/CD garantit la non-régression à chaque évolution du code et la stack Docker assure la reproductibilité de l'environnement sur toute machine. Les limites identifiées – dépendance à une clé API externe, absence de vérification email et absence de HTTPS en local – sont documentées et des recommandations de durcissement ont été formulées. La solution constitue une base solide pouvant évoluer vers une mise en production institutionnelle avec les adaptations appropriées.

Conclusion générale

Le projet *Chatbot USMS* a permis de concevoir et réaliser une application web intelligente dédiée à l'assistance universitaire des étudiants de l'ENSA Béni Mellal. Il combine frontend moderne multilingue, API robuste, base relationnelle, recherche vectorielle sémantique et modèle de langage large pour répondre à des questions académiques en langage naturel dans trois langues.

Le travail réalisé couvre l'analyse, la conception, l'implémentation d'une architecture en services spécialisés, la migration PostgreSQL, la gestion du contenu documentaire, la sécurisation à plusieurs niveaux, la protection des données, les tests automatisés et la préparation au déploiement. L'ensemble représente plus de 6 000 lignes de code, plus de 30 endpoints API et 127 tests automatisés validés en intégration continue.

Parmi les perspectives, on peut citer l'export .ics des emplois du temps, l'amélioration du workflow d'import FAQ avec prévisualisation avant indexation, l'ajout de notifications push pour les mises à jour de planning, le déploiement sur serveur institutionnel avec HTTPS, l'élargissement de la base de connaissances, la mise en place d'une authentification double facteur pour l'administrateur et l'intégration d'un audit de sécurité OWASP.

Bibliographie

- [1] FastAPI, *FastAPI Documentation*. Disponible sur : <https://fastapi.tiangolo.com/>. Consulté en mai 2026.
- [2] Vercel, *Next.js Documentation*. Disponible sur : <https://nextjs.org/docs>. Consulté en mai 2026.
- [3] PostgreSQL Global Development Group, *PostgreSQL Documentation*. Disponible sur : <https://www.postgresql.org/docs/>. Consulté en mai 2026.
- [4] Qdrant, *Qdrant Documentation*. Disponible sur : <https://qdrant.tech/documentation/>. Consulté en mai 2026.
- [5] Groq, *GroqCloud Documentation*. Disponible sur : <https://console.groq.com/docs>. Consulté en mai 2026.
- [6] SQLAlchemy, *SQLAlchemy Documentation*. Disponible sur : <https://docs.sqlalchemy.org/>. Consulté en mai 2026.
- [7] Alembic, *Alembic Documentation*. Disponible sur : <https://alembic.sqlalchemy.org/>. Consulté en mai 2026.
- [8] OWASP Foundation, *OWASP Top Ten*. Disponible sur : <https://owasp.org/www-project-top-ten/>. Consulté en mai 2026.
- [9] OWASP Foundation, *OWASP Top 10 for Large Language Model Applications*. Disponible sur : <https://owasp.org/www-project-top-10-for-large-language-model-applications/>. Consulté en mai 2026.
- [10] National Institute of Standards and Technology, *AI Risk Management Framework*. Disponible sur : <https://www.nist.gov/itl/ai-risk-management-framework>. Consulté en mai 2026.
- [11] Es, Shahul et al., *RAGAS : Automated Evaluation of Retrieval Augmented Generation*. Disponible sur : <https://arxiv.org/abs/2309.15217>. Consulté en mai 2026.
- [12] Redis Ltd, *Redis Documentation*. Disponible sur : <https://redis.io/docs/>. Consulté en mai 2026.
- [13] GitHub Inc., *GitHub Actions Documentation*. Disponible sur : <https://docs.github.com/en/actions>. Consulté en mai 2026.
- [14] Royaume du Maroc, *Loi n° 09-08 relative à la protection des personnes physiques à l'égard du traitement des données à caractère personnel*. Référence institutionnelle CNDP. Consulté en mai 2026.
- [15] Union Européenne, *Règlement (UE) 2016/679 – Règlement Général sur la Protection des Données*. Disponible sur : <https://gdpr-info.eu/>. Consulté en mai 2026.

Annexes

I. Arborescence simplifiée du projet

```
1 Chatbot-USMS/
2   backend/
3     main.py
4     routes/          # auth.py, chat.py, admin.py
5     services/        # auth, db, rag, groq, cache, chatbot...
6     models/          # schemas.py, db_models.py
7     scripts/         # bootstrap, seed, rebuild, benchmark
8     data/faq/         # Documents FAQ indexes (PDF, JSON, TXT)
9     data/seeds/       # timetables.json
10    data/logs/        # app.log (Loguru)
11    alembic/          # Migrations de schema
12    tests/            # 8 modules pytest + conftest.py
13 frontend/
14   src/app/           # Pages Next.js (login, chat, admin, profil,
15                       privacy)
16   src/components/    # Composants réutilisables (chat, admin,
17                       providers)
18   src/lib/           # api.ts, i18n.ts
19   src/store/         # useAuthStore.ts, useChatStore.ts
20   src/locales/       # fr.json, en.json, ar.json
21 docker-compose.yml
dev.ps1 / dev.sh
README.md
```

II. Livrables du projet

- code source backend FastAPI avec services, routes, modèles, migrations et tests automatisés ;
- code source frontend Next.js avec pages étudiant, profil, administration et composants réutilisables ;
- configuration Docker Compose pour PostgreSQL, Redis et Qdrant ;
- scripts de démarrage dev.ps1 et dev.sh et scripts de gestion des données ;
- fichiers FAQ et données d’emplois du temps utilisés pour la démonstration ;
- rapport PDF du projet avec captures réelles de l’application.

III. Résumé des endpoints API principaux

TABLE 4 – Principaux endpoints de l’API REST

Méthode	Route	Description
POST	/api/v1/auth/login	Connexion, retourne JWT et cookies de session.
POST	/api/v1/auth/register	Inscription étudiant avec consentement et confidentialité.
POST	/api/v1/auth/refresh	Renouvellement du jeton d’accès via cookie.
POST	/api/v1/auth/logout	Invalidation de la session.
GET	/api/v1/auth/me	Profil de l’utilisateur connecté.
POST	/api/v1/auth/me/language	Mise à jour de la langue préférée.
GET	/api/v1/auth/me/activity	Journal d’activité de l’utilisateur.
GET	/api/v1/auth/me/export	Export JSON de toutes les données personnelles (RGPD art. 20).
DELETE	/api/v1/auth/me	Suppression du compte et de toutes les données (RGPD art. 17).
POST	/api/v1/chat/stream	Question au chatbot (réponse en streaming SSE, 10 req/min).
POST	/api/v1/chat/feedback/{id}	Feedback sur un message (-1/0/1).
GET	/api/v1/conversations	Historique des conversations.
DELETE	/api/v1/conversations/{id}	Suppression d’une conversation.
POST	/api/v1/admin/upload/faq	Import d’un document FAQ (admin).
POST	/api/v1/admin/faq/rebuild	Reconstruction de l’index vectoriel (admin).
POST	/api/v1/admin/cache/flush	Vidage du cache Redis (admin).
POST	/api/v1/admin/data/cleanup	Nettoyage des données de plus de 180 jours (admin).
POST	/api/v1/admin/edt/publish	Publication d’un emploi du temps (admin).
GET	/api/v1/admin/stats	Statistiques globales de la plateforme (admin).
GET	/healthz	Sonde de disponibilité légère.